Multifamily Property Performance — Automated Reporting

Google Sheets + Apps Script sample · William L. · GA4 + Google Business Profile + weekly Yardi → database → dashboard

> Sample with mock data for a 3-property portfolio. On your project I'd apply these exact patterns to your real spreadsheet, GA4 property, and GBP locations.

Architecture

Weekly Yardi sheet (no API) → Apps Script parser (header-by-name, name-normalize, TOTAL reconcile) —+— **GA4** (Data API, service account) + **Google Business Profile** (Performance API once approved, or connector bridge) → **History** (append-only database, key = week_ending | property_id, idempotent upsert) → **Dashboard** (Sheets now, Looker Studio optional) + **Review** tab for anything unreconciled + **weekly time-trigger** + anomaly alerts.

Header-by-name Yardi parse · Canonical name normalization · TOTAL-row reconciliation · Idempotent upsert (composite key) · Fatal-vs-warning validation · Preserves GA4/GBP on re-import · GA4 Data API (service account) · GBP Performance API / connector · Scheduled refresh · WoW anomaly alerts · Self-tests · Codex-audited

The dashboard (live from the History database)

Multifamily Property Performance

Live from the History database — GA4, Google Business Profile, and the weekly Yardi report, consolidate

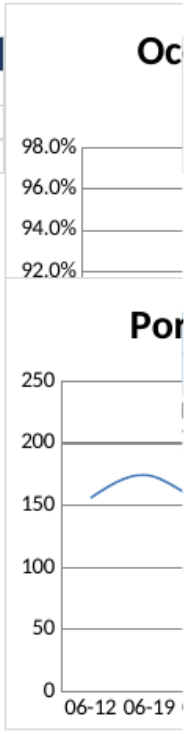
Latest week ending: **2026-08-28** 2026-08-21 *(prior week)*

Portfolio occupancy		Leased %		Total leads
92.2%		**94.1%**		**188**
+0.9%	*WoW*	*+1.3%*	*WoW*	*+10.6%*

Conversions		GBP calls		Website sessions
141		**379**		**2,880**
-13.0%	*WoW*	*+11.5%*	*WoW*	*-10.4%*

Same-store property comparison (latest week)

Property	Occupancy	Leased %	Move-ins
Maple Grove Apartments	90.0%	91.3%	9
Riverside Commons	92.1%	95.3%	4
Oak Park Residences	96.8%	98.1%	7



Portfolio trend (last 12 weeks)

Week	Occupancy	Leads
06-12	90.4%	156
06-19	91.0%	174
06-26	91.7%	152
07-03	91.0%	169
07-10	91.7%	186
07-17	90.7%	163
07-24	91.4%	181
07-31	90.7%	199
08-07	91.4%	176
08-14	92.2%	193
08-21	91.3%	170
08-28	92.2%	188

How it maps to your scope

You asked for	In this sample
Review sheet + build a supporting database	History tab (append-only, keyed, idempotent)
Automate GA4 + GBP pulls	GA4 Data API (service account) + GBP seam (API/connector) folding into History
Parse weekly Yardi, handle changing row counts	Header-by-name parser + normalization + reconciliation, changing rows handled
Maintain history, MoM / period trends	12-week History + trend + T-3/T-12 views
Clean dashboard, scheduled refresh	Dashboard tab + weekly time-driven trigger
Test values against source; flag what can't be automated	Fatal/warning validation + TOTAL reconciliation, routed to Review
Property-to-property comparison	Same-store comparison table + occupancy chart

The code

`00_Config.gs`

```
/**
 * 00_Config.gs — single source of truth.
 * Sheet names, the Yardi header map (by NAME, never by column position), the
 * canonical property lookup, and thresholds. Secrets (GA4 property id, service-
 * account key, any connector token) live in Script Properties, never in code.
 *
 * Multifamily reporting automation — sample by William L.
 */
const CONFIG = Object.freeze({
  VERSION: '1.0.0',
  DRY_RUN: false,                    // true = log instead of writing / emailing

  SHEETS: {
    HISTORY:  'History',             // append-only database (composite key below)
    YARDI:    'Raw_Yardi_Weekly',
    GA4:      'Raw_GA4',
    GBP:      'Raw_GBP',
    REVIEW:   'Review',              // exceptions land here, never silently dropped
    DASHBOARD:'Dashboard',
  },

  // Composite key that makes ingestion idempotent (re-run a week -> upsert, not dupe)
  KEY: ['week_ending', 'property_id'],

  // Columns the Yardi import owns. On a re-import of an existing week we update
  // ONLY these, so the GA4/GBP columns written by the external merge survive.
  YARDI_OWNED: [
    'Week Ending', 'Property ID', 'Property', 'Units', 'Occupied', 'Leased',
    'Move-ins', 'Move-outs', 'Delinquencies', 'Applications',
  ],

  // History column order (must match the sheet header row).
  HISTORY_HEADERS: [
    'Week Ending','Property ID','Property','Units','Occupied','Leased',
    'Move-ins','Move-outs','Delinquencies','Applications',
    'Sessions','Users','Conversions','Leads',
    'GBP Calls','GBP Website Clicks','GBP Directions','GBP Views',
  ],

  // Yardi report is messy: header is not on row 1, names drift, a TOTAL row trails.
  // We locate the header by matching these known labels, then map by name.
  YARDI: {
    HEADER_MATCH: ['property', 'units', 'occupied'],   // row containing all of these = header
    // report label (lowercased, trimmed) -> our History field
    FIELD_MAP: {
      'property': 'property_name',
      'units': 'Units',
      'occupied': 'Occupied',
      'leased': 'Leased',
      'move in': 'Move-ins',
      'move out': 'Move-outs',
      'delinq.': 'Delinquencies',
      'delinquencies': 'Delinquencies',
      'applications': 'Applications',
    },
    TOTAL_LABELS: ['total', 'grand total', 'portfolio total'],  // skip these rows
  },

  // Canonical property directory — normalises drifting names ("Oak Park Res." ->
  // "Oak Park Residences") and supplies the stable property_id used in the key.
  PROPERTIES: [
    { id: 'P1', canonical: 'Maple Grove Apartments', aliases: ['maple grove apartments','maple grove'] },
    { id: 'P2', canonical: 'Riverside Commons',       aliases: ['riverside commons','riverside'] },
    { id: 'P3', canonical: 'Oak Park Residences',     aliases: ['oak park residences','oak park res.','oak park res','oak
park'] },
  ],

  // Validation thresholds
  ROWCOUNT_TOLERANCE: 0.30,          // flag if this week's property count swings >30% vs last
  ANOMALY_DROP: 0.20,                // flag a metric that drops >20% week-over-week

  PROP_KEYS: Object.freeze({
    OPS_EMAIL:      'OPS_EMAIL',      // digest / anomaly alerts
    GA4_PROPERTY:   'GA4_PROPERTY_ID',
```

```
    GA4_SA_KEY:     'GA4_SERVICE_ACCOUNT_KEY',   // JSON key, Script Properties only
  }),
});

function getProp_(k, d) {
  const v = PropertiesService.getScriptProperties().getProperty(k);
  return (v === null || v === undefined) ? d : v;
}
```

10_Yardi.gs

```
/**
 * 10_Yardi.gs — the showpiece: parse the weekly Yardi report robustly.
 *
 * The report has NO API, its header is not on row 1, property names drift week to
 * week, and it carries a trailing TOTAL row. This reads the whole used range,
 * finds the header BY NAME, maps columns by name, normalises each property to its
 * canonical name + stable id, reconciles the property rows against the report's
 * own TOTAL row, and upserts into the History database by composite key so a
 * re-run of the same week updates rather than duplicates. Anything it can't
 * reconcile is routed to the Review sheet — never silently ingested.
 */

/**
 * Entry point: import the current weekly Yardi tab into History.
 * Returns the imported week (Date) on success, or null on a fatal failure — so
 * the caller (weeklyRefresh) knows exactly which week to merge/scan and can abort
 * cleanly. Caller holds the lock for the whole weekly transaction.
 */
function importYardiWeek_() {
  const ss = SpreadsheetApp.getActive();
  const src = mustSheet_(ss, CONFIG.SHEETS.YARDI);
  const grid = src.getDataRange().getValues();

  const weekEnding = findWeekEnding_(grid);
  if (!weekEnding) { flagReview_('', '(import)', 'parse error', 'Could not find a "Week ending" date in the Yardi report — nothing imported.'); return null; }

  const hdr = findHeaderRow_(grid);
  if (hdr < 0) { flagReview_(weekEnding, '(import)', 'parse error', 'Could not locate the Yardi header row by name — nothing imported.'); return null; }

  const colOf = mapColumns_(grid[hdr]);
  const parsed = [];
  const seenKeys = {};
  let reportTotal = null;
  let dupFatal = false;

  for (let r = hdr + 1; r < grid.length; r++) {
    const rawName = String(grid[r][colOf.property_name] || '').trim();
    if (!rawName) continue;                                  // blank spacer row
    if (isTotalRow_(rawName)) { reportTotal = readRow_(grid[r], colOf); continue; }

    const prop = matchProperty_(rawName);
    if (!prop) { flagReview_(weekEnding, rawName, 'unknown property', 'Not in the canonical directory; row skipped.'); continue; }

    const rec = readRow_(grid[r], colOf);
    rec.week_ending = weekEnding;
    rec.property_id = prop.id;
    rec.Property = prop.canonical;
    if (rawName !== prop.canonical) {
      flagReview_(weekEnding, rawName, 'name normalized', 'Matched "' + rawName + '" -> "' + prop.canonical + '".');
    }
    const k = keyOf_(weekEnding, prop.id);
    if (seenKeys[k]) {
      dupFatal = true;
      flagReview_(weekEnding, prop.canonical, 'duplicate row', 'Property appears more than once in this week\'s report — import aborted.');
      continue;
    }
    seenKeys[k] = true;
    parsed.push(rec);
  }

  const result = validateWeek_(weekEnding, parsed, reportTotal);
  if (dupFatal || result.fatal.length) {
    flagReview_(weekEnding, '(import)', 'ABORTED', (result.fatal.length + (dupFatal ? 1 : 0)) + ' fatal issue(s) — nothing written to History this run.');
    logInfo_('Yardi import ' + fmtDate_(weekEnding) + ' ABORTED (' + result.fatal.length + ' fatal).');
    return null;
  }
  const n = upsertHistory_(parsed);
  logInfo_('Yardi import ' + fmtDate_(weekEnding) + ': ' + n + ' rows upserted (' + result.warnings.length + ' warning(s)).');
```

```
  return weekEnding;
}

/* ---------- parsing helpers (pure where possible) ---------- */

function findWeekEnding_(grid) {
  for (let r = 0; r < Math.min(grid.length, 12); r++) {
    for (let c = 0; c < grid[r].length; c++) {
      if (String(grid[r][c]).trim().toLowerCase() === 'week ending') {
        const v = grid[r][c + 1];
        return v instanceof Date ? startOfDay_(v) : parseDate_(v);
      }
    }
  }
  return null;
}

function findHeaderRow_(grid) {
  const need = CONFIG.YARDI.HEADER_MATCH;
  for (let r = 0; r < grid.length; r++) {
    const cells = grid[r].map(function (v) { return String(v).trim().toLowerCase(); });
    if (need.every(function (label) { return cells.indexOf(label) !== -1; })) return r;
  }
  return -1;
}

/** Map each report label to a History field via CONFIG.YARDI.FIELD_MAP (by NAME). */
function mapColumns_(headerRow) {
  const map = {};
  headerRow.forEach(function (label, idx) {
    const key = String(label).trim().toLowerCase();
    const field = CONFIG.YARDI.FIELD_MAP[key];
    if (field) map[field] = idx;
  });
  return map;
}

function readRow_(row, colOf) {
  const rec = {};
  Object.keys(colOf).forEach(function (field) {
    const v = row[colOf[field]];
    rec[field] = (field === 'property_name') ? String(v).trim() : num_(v);
  });
  return rec;
}

function isTotalRow_(name) {
  const n = name.trim().toLowerCase();
  return CONFIG.YARDI.TOTAL_LABELS.indexOf(n) !== -1;
}

/** Normalise a drifting property name to {id, canonical} or null. */
function matchProperty_(rawName) {
  const n = rawName.trim().toLowerCase();
  return CONFIG.PROPERTIES.find(function (p) {
    return p.canonical.toLowerCase() === n || p.aliases.indexOf(n) !== -1;
  }) || null;
}

/* ---------- validation ---------- */

/**
 * Returns {fatal:[], warnings:[]}. Fatal issues abort the whole write (bad data
 * must not reach History); warnings are surfaced but don't block.
 */
function validateWeek_(weekEnding, parsed, reportTotal) {
  const fatal = [], warnings = [];
  const warn = function (entity, check, detail) { warnings.push(check); flagReview_(weekEnding, entity, check, detail); };
  const fail = function (entity, check, detail) { fatal.push(check); flagReview_(weekEnding, entity, check, detail); };

  // 1. count sanity vs the most recent prior week — compare UNIQUE property ids.
  const prior = priorWeekCount_(weekEnding);
  const nUnique = parsed.length;   // already deduped by key in the caller
  if (prior > 0) {
    const swing = Math.abs(nUnique - prior) / prior;
```

```
    if (swing > CONFIG.ROWCOUNT_TOLERANCE) {
      warn('(row count)', 'count swing',
        nUnique + ' properties this week vs ' + prior + ' last week (' + Math.round(swing * 100) + '%) — verify
completeness.');
    }
  }
  // 2. required fields + impossible values (impossible = fatal for that row's week)
  parsed.forEach(function (rec) {
    ['Units', 'Occupied'].forEach(function (f) {
      if (!Number.isFinite(rec[f])) fail(rec.Property, 'missing value', 'Field "' + f + '" is blank/non-numeric.');
    });
    if (Number.isFinite(rec.Occupied) && Number.isFinite(rec.Units) && rec.Occupied > rec.Units) {
      fail(rec.Property, 'impossible value', 'Occupied (' + rec.Occupied + ') > Units (' + rec.Units + ').');
    }
  });
  // 3. reconcile to the report's own TOTAL row. A present-but-mismatched total is
  //    fatal; a missing total is a warning (can't verify).
  if (reportTotal && Number.isFinite(reportTotal.Units)) {
    const allFinite = parsed.every(function (r) { return Number.isFinite(r.Units); });
    const sum = parsed.reduce(function (a, r) { return a + (Number.isFinite(r.Units) ? r.Units : 0); }, 0);
    if (!allFinite) {
      fail('(report total)', 'reconciliation FAILED', 'A property Units value is non-numeric, so the total cannot be
reconciled.');
    } else if (sum !== reportTotal.Units) {
      fail('(report total)', 'reconciliation FAILED', 'Sum of property units (' + sum + ') != Yardi TOTAL (' +
reportTotal.Units + ').');
    } else {
      flagReview_(weekEnding, '(report total)', 'reconciliation OK', 'Sum of property rows == Yardi TOTAL (units ' + sum +
').');
    }
  } else {
    warn('(report total)', 'no total row', 'No usable TOTAL row found — imported without a reconciliation check.');
  }
  return { fatal: fatal, warnings: warnings };
}

/* ---------- History upsert (idempotent by composite key) ---------- */

function upsertHistory_(records) {
  const ss = SpreadsheetApp.getActive();
  const sheet = mustSheet_(ss, CONFIG.SHEETS.HISTORY);
  const headers = CONFIG.HISTORY_HEADERS;
  const owned = CONFIG.YARDI_OWNED;             // columns Yardi may overwrite
  const last = sheet.getLastRow();
  const existing = last > 1 ? sheet.getRange(2, 1, last - 1, headers.length).getValues() : [];

  const idx = {};
  existing.forEach(function (row, i) { idx[keyOf_(row[0], row[1])] = i; });

  const applyOwned = function (row, rec) {
    headers.forEach(function (h, c) { if (owned.indexOf(h) !== -1) row[c] = histValue_(h, rec); });
  };
  const toAppend = [];
  const staged = {};            // key -> index into toAppend
  records.forEach(function (rec) {
    const k = keyOf_(rec.week_ending, rec.property_id);
    if (idx[k] !== undefined) {
      applyOwned(existing[idx[k]], rec);                // update owned cols, keep GA4/GBP
    } else if (staged[k] !== undefined) {
      applyOwned(toAppend[staged[k]], rec);             // same key twice this run -> merge, don't re-append
    } else {
      const rowArr = headers.map(function (h) { return histValue_(h, rec); });
      staged[k] = toAppend.push(rowArr) - 1;
    }
  });

  if (existing.length) sheet.getRange(2, 1, existing.length, headers.length).setValues(existing);
  if (toAppend.length) sheet.getRange(sheet.getLastRow() + 1, 1, toAppend.length, headers.length).setValues(toAppend);
  return records.length;
}

function histValue_(header, rec) {
  switch (header) {
    case 'Week Ending': return rec.week_ending;
    case 'Property ID': return rec.property_id;
```

```
    case 'Property':    return rec.Property;
    default:            return Number.isFinite(rec[header]) ? rec[header] : '';
  }
}

function keyOf_(week, propId) {
  const w = week instanceof Date ? fmtDate_(week) : String(week);
  return w + '|' + String(propId);
}

/** Count of UNIQUE properties in the most recent week strictly before weekEnding. */
function priorWeekCount_(weekEnding) {
  const sheet = mustSheet_(SpreadsheetApp.getActive(), CONFIG.SHEETS.HISTORY);
  const last = sheet.getLastRow();
  if (last < 2) return 0;
  const rows = sheet.getRange(2, 1, last - 1, 2).getValues();   // [week, property_id]
  const target = weekEnding.getTime();
  let bestTime = -1;
  rows.forEach(function (r) {
    if (r[0] instanceof Date && r[0].getTime() < target && r[0].getTime() > bestTime) bestTime = r[0].getTime();
  });
  if (bestTime < 0) return 0;
  const ids = {};
  rows.forEach(function (r) { if (r[0] instanceof Date && r[0].getTime() === bestTime) ids[r[1]] = true; });
  return Object.keys(ids).length;
}
```

20_ExternalPulls.gs

```
/**
 * 20_ExternalPulls.gs — GA4 + Google Business Profile automated pulls.
 *
 * These are the two API integrations. In this sample they run against mock
 * "Raw_GA4" / "Raw_GBP" tabs so the whole pipeline is demonstrable without live
 * credentials; the functions are structured exactly as the production versions,
 * with the real API calls shown and gated behind Script-Property credentials.
 *
 * GA4:  GA4 Data API v1 (runReport) via a service account shared on the property
 *       — no OAuth consent screen to maintain. Free, well within quota for a few
 *       properties on a weekly cadence.
 * GBP:  Business Profile Performance API (the old GMB API was split into 8 APIs).
 *       That API is ACCESS-GATED (formal request + 60-day-verified profile +
 *       Google's manual review), so production has two modes: (a) direct API once
 *       access is granted, or (b) a connector bridge that writes GBP metrics into
 *       Raw_GBP under its own approved app, so the schedule is never blocked by
 *       Google's review queue. This code reads Raw_GBP either way.
 */

/**
 * Refresh both external feeds, then fold them into History for `week`.
 * `week` is passed by weeklyRefresh (the week that was just imported) so a
 * backfill of an OLDER week merges into that week, not the newest one. The menu
 * action calls it with no argument, where it targets the latest week.
 */
function refreshAll(week) {
  pullGA4_();          // no-op in mock mode unless credentials are set
  pullGBP_();          // connector or API writes Raw_GBP; here it's mock
  mergeExternalIntoHistory_(week);
}

/**
 * GA4 Data API pull. Production path shown; runs only when GA4_PROPERTY_ID +
 * a service-account key are present in Script Properties, else leaves the mock
 * Raw_GA4 tab in place so the demo still works.
 */
function pullGA4_() {
  const propertyId = getProp_(CONFIG.PROP_KEYS.GA4_PROPERTY, '');
  const saKey = getProp_(CONFIG.PROP_KEYS.GA4_SA_KEY, '');
  if (!propertyId || !saKey) {
    logInfo_('GA4: no credentials set — using mock Raw_GA4 (production uses the GA4 Data API).');
    return;
  }
  try {
    const token = serviceAccountToken_(JSON.parse(saKey),
      'https://www.googleapis.com/auth/analytics.readonly');
    const url = 'https://analyticsdata.googleapis.com/v1beta/properties/' + propertyId + ':runReport';
    // The property model is set per deployment: either one GA4 property per managed
    // property (loop property ids), or a single property with a dimension that
    // identifies the property (e.g. a custom dimension / hostname). Shape the
    // report to whatever your GA4 setup uses; this is the integration seam.
    const body = {
      dateRanges: [{ startDate: '7daysAgo', endDate: 'yesterday' }],
      dimensions: [{ name: 'hostName' }],          // placeholder — swap to your property identifier
      metrics: [{ name: 'sessions' }, { name: 'totalUsers' }, { name: 'conversions' }],
    };
    const res = UrlFetchApp.fetch(url, {
      method: 'post', contentType: 'application/json',
      headers: { Authorization: 'Bearer ' + token },
      payload: JSON.stringify(body), muteHttpExceptions: true,
    });
    if (res.getResponseCode() !== 200) {
      flagReview_(new Date(), '(GA4)', 'api error', 'GA4 Data API returned HTTP ' + res.getResponseCode());
      return;
    }
    const wrote = writeGA4Rows_(JSON.parse(res.getContentText()));   // maps response -> Raw_GA4
    if (wrote === 0) {
      flagReview_(new Date(), '(GA4)', 'not persisted',
        'GA4 Data API responded but no rows were mapped to Raw_GA4 — finalize the property mapping for this account.');
      return;
    }
    logInfo_('GA4: wrote ' + wrote + ' property row(s) to Raw_GA4.');
  } catch (err) {
    flagReview_(new Date(), '(GA4)', 'api error', String(err.message));
```

```
  }
}

/**
 * GBP metrics land in Raw_GBP — from the Business Profile Performance API once
 * access is approved, or from a connector in the meantime. This function is the
 * seam; the demo ships with a mock Raw_GBP already populated.
 */
function pullGBP_() {
  // Production: iterate locations, call the Performance API's fetchMultiDailyMetricsTimeSeries
  // for CALL_CLICKS, WEBSITE_CLICKS, BUSINESS_DIRECTION_REQUESTS, BUSINESS_IMPRESSIONS_*.
  // (Photo-view and search-term-split metrics were deprecated, so we don't request them.)
  logInfo_('GBP: reading Raw_GBP (API once access is approved, or connector bridge until then).');
}

/**
 * Fold the current Raw_GA4 + Raw_GBP snapshots into History for the latest week,
 * keyed by property. Idempotent — reuses the same upsert path as the Yardi import.
 */
function mergeExternalIntoHistory_(week) {
  const ss = SpreadsheetApp.getActive();
  week = week || latestHistoryWeek_(ss);           // explicit week wins (backfill-safe)
  if (!week) { logInfo_('External merge skipped: no Yardi week in History yet.'); return; }

  const ga4 = readNamedTable_(ss, CONFIG.SHEETS.GA4);   // {property -> {Sessions,Users,Conversions,Leads}}
  const gbp = readNamedTable_(ss, CONFIG.SHEETS.GBP);   // {property -> {Calls,...}}

  const patches = [];
  CONFIG.PROPERTIES.forEach(function (p) {
    const g = ga4[p.canonical] || {};
    const b = gbp[p.canonical] || {};
    patches.push({
      week_ending: week, property_id: p.id, Property: p.canonical,
      Sessions: g['Sessions'], Users: g['Users'], Conversions: g['Conversions'], Leads: g['Leads'],
      'GBP Calls': b['Calls'], 'GBP Website Clicks': b['Website Clicks'],
      'GBP Directions': b['Directions'], 'GBP Views': b['Views'],
    });
  });
  patchHistory_(patches);      // update only the columns supplied, keep Yardi fields intact
  logInfo_('External merge: GA4 + GBP folded into History for ' + fmtDate_(week) + '.');
}
```

30_Anomaly.gs

```
/**
 * 30_Anomaly.gs — the above-and-beyond that property managers actually act on.
 * After each refresh, compare the latest week to the prior week per property and
 * flag any monitored metric that dropped more than CONFIG.ANOMALY_DROP. Flags go
 * to the Review sheet and (optionally) an email digest — managers care about a
 * property's calls falling off far more than another historical chart.
 */

const MONITORED = ['GBP Calls', 'GBP Directions', 'Leads', 'Sessions'];

function detectAnomalies(week) {
  const ss = SpreadsheetApp.getActive();
  const latest = week || latestHistoryWeek_(ss);   // scan the week just refreshed (backfill-safe)
  if (!latest) return [];
  const prior = priorWeekDate_(ss, latest);
  if (!prior) { logInfo_('Anomaly scan: no prior week to compare.'); return []; }

  const cur = weekByProperty_(ss, latest);
  const prev = weekByProperty_(ss, prior);
  const found = [];

  CONFIG.PROPERTIES.forEach(function (p) {
    const a = cur[p.id], b = prev[p.id];
    if (!a || !b) return;
    MONITORED.forEach(function (m) {
      const now = Number(a[m]), was = Number(b[m]);
      if (!Number.isFinite(now) || !Number.isFinite(was) || was <= 0) return;
      const drop = (was - now) / was;
      if (drop > CONFIG.ANOMALY_DROP) {
        const pct = Math.round(drop * 100);
        flagReview_(latest, p.canonical, 'anomaly flag',
          m + ' down ' + pct + '% WoW (' + was + ' -> ' + now + ') — surfaced for review.');
        found.push({ property: p.canonical, metric: m, dropPct: pct, was: was, now: now });
      }
    });
  });

  if (found.length) emailAnomalies_(latest, found);
  logInfo_('Anomaly scan: ' + found.length + ' flag(s).');
  return found;
}

function emailAnomalies_(week, found) {
  const to = getProp_(CONFIG.PROP_KEYS.OPS_EMAIL, '');
  if (!to) return;
  const rows = found.map(function (f) {
    return '<tr><td>' + esc_(f.property) + '</td><td>' + esc_(f.metric) +
      '</td><td align="right" style="color:#7A1F0A">-' + f.dropPct + '%</td>' +
      '<td align="right">' + f.was + ' &rarr; ' + f.now + '</td></tr>';
  }).join('');
  const html =
    '<div style="font-family:Arial,sans-serif;max-width:560px">' +
    '<h2 style="color:#0F766E">Performance anomalies — week ending ' + fmtDate_(week) + '</h2>' +
    '<table cellpadding="6" style="border-collapse:collapse;width:100%">' +
    '<tr><th align="left">Property</th><th align="left">Metric</th><th>WoW</th><th>Change</th></tr>' +
    rows + '</table></div>';
  sendMail_(to, 'Property performance anomalies (' + found.length + ') — ' + fmtDate_(week), html);
}
```

40_Utils_Menu_Setup.gs

```
/**
 * 40_Utils_Menu_Setup.gs — installer, menu, shared helpers, and self-tests.
 */

/* ---------- menu + scheduled entry point ---------- */

function onOpen() {
  SpreadsheetApp.getUi().createMenu('📊 Reporting')
    .addItem('Run setup / repair', 'setup')
    .addSeparator()
    .addItem('Import Yardi week', 'importYardiWeek')
    .addItem('Refresh GA4 + GBP', 'refreshAll')
    .addItem('Run full weekly refresh', 'weeklyRefresh')
    .addItem('Scan for anomalies', 'detectAnomalies')
    .addSeparator()
    .addItem('Run self-tests', 'runTests')
    .addToUi();
}

/** Public, locked Yardi import (menu + standalone). Returns the imported week. */
function importYardiWeek() {
  const lock = LockService.getScriptLock();
  lock.waitLock(30 * 1000);
  try { return importYardiWeek_(); } finally { lock.releaseLock(); }
}

/**
 * The scheduled job: one lock around the WHOLE weekly transaction so a manual and
 * a scheduled run can't interleave and clobber History. Aborts cleanly if the
 * Yardi import fails, so external metrics are never merged into the wrong week.
 */
function weeklyRefresh() {
  const lock = LockService.getScriptLock();
  lock.waitLock(5 * 60 * 1000);
  try {
    const week = importYardiWeek_();
    if (!week) { logInfo_('Weekly refresh ABORTED — Yardi import failed; external merge + anomaly scan skipped.'); return; }
    refreshAll(week);          // merge GA4/GBP into the exact week imported (backfill-safe)
    detectAnomalies(week);     // scan that same week
    logInfo_('Weekly refresh complete for ' + fmtDate_(week) + '.');
  } finally { lock.releaseLock(); }
}

/** Idempotent installer: ensures tabs exist and the weekly trigger is installed once. */
function setup() {
  const ss = SpreadsheetApp.getActive();
  Object.values(CONFIG.SHEETS).forEach(function (name) {
    if (!ss.getSheetByName(name)) ss.insertSheet(name);
  });
  const history = ss.getSheetByName(CONFIG.SHEETS.HISTORY);
  if (history.getLastRow() === 0) {
    history.appendRow(CONFIG.HISTORY_HEADERS);
    history.getRange(1, 1, 1, CONFIG.HISTORY_HEADERS.length).setFontWeight('bold');
    history.setFrozenRows(1);
  }
  ensureTrigger_('weeklyRefresh', function () {
    return ScriptApp.newTrigger('weeklyRefresh').timeBased().everyWeeks(1)
      .onWeekDay(ScriptApp.WeekDay.MONDAY).atHour(6).create();
  });
  logInfo_('setup() complete (v' + CONFIG.VERSION + ').');
  SpreadsheetApp.getActive().toast('Setup complete.', 'Reporting', 4);
}

function ensureTrigger_(handler, createFn) {
  const exists = ScriptApp.getProjectTriggers().some(function (t) { return t.getHandlerFunction() === handler; });
  if (!exists) createFn();
}

/* ---------- Review + logging ---------- */

function flagReview_(week, entity, check, detail) {
  try {
    const ss = SpreadsheetApp.getActive();
    const sheet = ss.getSheetByName(CONFIG.SHEETS.REVIEW) || ss.insertSheet(CONFIG.SHEETS.REVIEW);
```

```
    if (sheet.getLastRow() === 0) sheet.appendRow(['Week Ending', 'Property', 'Check', 'Detail']);
    sheet.appendRow([week || '', entity || '', check || '', detail || '']);
  } catch (err) { console.error('flagReview_ failed: ' + err.message); }
}

function logInfo_(msg) { console.log('[' + CONFIG.VERSION + '] ' + msg); }

function sendMail_(to, subject, html) {
  if (!to) return;
  if (CONFIG.DRY_RUN) { logInfo_('[DRY_RUN] would email: ' + subject); return; }
  try { MailApp.sendEmail({ to: to, subject: subject, htmlBody: html }); }
  catch (err) { console.error('sendMail_ failed: ' + err.message); }
}

/* ---------- History read helpers ---------- */

function latestHistoryWeek_(ss) {
  const sheet = mustSheet_(ss, CONFIG.SHEETS.HISTORY);
  const last = sheet.getLastRow();
  if (last < 2) return null;
  const weeks = sheet.getRange(2, 1, last - 1, 1).getValues();
  let best = null;
  weeks.forEach(function (r) { if (r[0] instanceof Date && (!best || r[0] > best)) best = r[0]; });
  return best;
}

function priorWeekDate_(ss, latest) {
  const sheet = mustSheet_(ss, CONFIG.SHEETS.HISTORY);
  const last = sheet.getLastRow();
  const weeks = sheet.getRange(2, 1, last - 1, 1).getValues();
  let best = null;
  weeks.forEach(function (r) {
    if (r[0] instanceof Date && r[0] < latest && (!best || r[0] > best)) best = r[0];
  });
  return best;
}

/** {property_id -> {header: value}} for one week. */
function weekByProperty_(ss, week) {
  const sheet = mustSheet_(ss, CONFIG.SHEETS.HISTORY);
  const headers = CONFIG.HISTORY_HEADERS;
  const last = sheet.getLastRow();
  const rows = last > 1 ? sheet.getRange(2, 1, last - 1, headers.length).getValues() : [];
  const out = {};
  rows.forEach(function (row) {
    if (row[0] instanceof Date && row[0].getTime() === week.getTime()) {
      const rec = {}; headers.forEach(function (h, i) { rec[h] = row[i]; });
      out[row[1]] = rec;                       // key by Property ID
    }
  });
  return out;
}

/** Read a small "Property + metrics" raw tab into {canonicalName -> {metric: value}}. */
function readNamedTable_(ss, name) {
  const sheet = mustSheet_(ss, name);
  const grid = sheet.getDataRange().getValues();
  const hdrRow = grid.findIndex(function (r) {
    return String(r[0]).trim().toLowerCase() === 'property';
  });
  if (hdrRow < 0) return {};
  const headers = grid[hdrRow].map(function (v) { return String(v).trim(); });
  const out = {};
  for (let r = hdrRow + 1; r < grid.length; r++) {
    const rawName = String(grid[r][0] || '').trim();
    if (!rawName) continue;
    const prop = matchProperty_(rawName);
    const key = prop ? prop.canonical : rawName;
    const rec = {};
    for (let c = 1; c < headers.length; c++) rec[headers[c]] = num_(grid[r][c]);
    out[key] = rec;
  }
  return out;
}
```

```
/** Patch only the supplied columns of matching History rows (keeps Yardi fields). */
function patchHistory_(patches) {
  const ss = SpreadsheetApp.getActive();
  const sheet = mustSheet_(ss, CONFIG.SHEETS.HISTORY);
  const headers = CONFIG.HISTORY_HEADERS;
  const last = sheet.getLastRow();
  if (last < 2) return;
  const rows = sheet.getRange(2, 1, last - 1, headers.length).getValues();
  const idx = {};
  rows.forEach(function (row, i) { idx[keyOf_(row[0], row[1])] = i; });
  patches.forEach(function (patch) {
    const i = idx[keyOf_(patch.week_ending, patch.property_id)];
    if (i === undefined) return;
    headers.forEach(function (h, c) {
      if (patch[h] !== undefined && patch[h] !== null && Number.isFinite(Number(patch[h]))) rows[i][c] = Number(patch[h]);
    });
  });
  sheet.getRange(2, 1, rows.length, headers.length).setValues(rows);
}

/**
 * Map a GA4 runReport response into Raw_GA4 (one row per canonical property) and
 * return the number of rows written. The row->property aggregation depends on the
 * client's GA4 setup (per-property properties, or a property-identifying
 * dimension), so this is finalized at deployment against the real account. In the
 * demo GA4 runs in mock mode, so this returns 0 and the mock Raw_GA4 stands.
 */
function writeGA4Rows_(apiResponse) {
  const rows = (apiResponse && apiResponse.rows) || [];
  logInfo_('GA4: response carried ' + rows.length + ' report rows (mapping finalized per your GA4 property model).');
  return 0;
}

/* ---------- service-account JWT (GA4) ---------- */

/** Mint an OAuth access token from a service-account key (RS256 JWT grant). */
function serviceAccountToken_(key, scope) {
  const now = Math.floor(Date.now() / 1000);
  const header = base64url_(JSON.stringify({ alg: 'RS256', typ: 'JWT' }));
  const claim = base64url_(JSON.stringify({
    iss: key.client_email, scope: scope, aud: 'https://oauth2.googleapis.com/token',
    iat: now, exp: now + 3600,
  }));
  const signature = Utilities.computeRsaSha256Signature(header + '.' + claim, key.private_key);
  const jwt = header + '.' + claim + '.' + base64url_(signature);
  const res = UrlFetchApp.fetch('https://oauth2.googleapis.com/token', {
    method: 'post', muteHttpExceptions: true,
    payload: { grant_type: 'urn:ietf:params:oauth:grant-type:jwt-bearer', assertion: jwt },
  });
  if (res.getResponseCode() !== 200) {
    throw new Error('Token endpoint returned HTTP ' + res.getResponseCode() + ' (check the service-account key/scope).');
  }
  const token = JSON.parse(res.getContentText()).access_token;
  if (!token) throw new Error('Token endpoint returned no access_token.');
  return token;
}

function base64url_(data) {
  const bytes = (typeof data === 'string') ? Utilities.newBlob(data).getBytes() : data;
  return Utilities.base64EncodeWebSafe(bytes).replace(/=+$/, '');
}

/* ---------- tiny pure helpers ---------- */

function mustSheet_(ss, name) {
  const s = ss.getSheetByName(name);
  if (!s) throw new Error('Missing sheet "' + name + '". Run setup() first.');
  return s;
}
function num_(v) {
  if (v === '' || v === null || v === undefined) return NaN;
  const n = Number(String(v).replace(/[$,%\s]/g, ''));
  return Number.isFinite(n) ? n : NaN;
}
function startOfDay_(d) { const x = new Date(d); x.setHours(0, 0, 0, 0); return x; }
```

```
function parseDate_(v) { const d = new Date(v); return isNaN(d.getTime()) ? null : startOfDay_(d); }
function fmtDate_(d) { return Utilities.formatDate(d, Session.getScriptTimeZone(), 'yyyy-MM-dd'); }
function esc_(s) { return String(s).replace(/&/g, '&amp;').replace(/</g, '&lt;').replace(/>/g, '&gt;'); }

/* ---------- self-tests (pure logic; run from the menu) ---------- */

function runTests() {
  const results = [];
  const ok = function (name, cond) { results.push((cond ? 'PASS' : 'FAIL') + ' — ' + name); };

  ok('num_ strips $ and commas', num_('$1,240') === 1240);
  ok('num_ blank -> NaN', Number.isNaN(num_('')));
  ok('canonical match exact', matchProperty_('Riverside Commons').id === 'P2');
  ok('canonical match drifted name', matchProperty_('Oak Park Res.').id === 'P3');
  ok('canonical match trailing space', matchProperty_('Maple Grove Apartments ').id === 'P1');
  ok('unknown property -> null', matchProperty_('Somewhere Else') === null);
  ok('total row detected', isTotalRow_('TOTAL') && isTotalRow_(' grand total '));
  ok('total row not a property', !isTotalRow_('Maple Grove Apartments'));

  const hdr = ['Property', 'Units', 'Occupied', 'Leased', 'Move In', 'Some Extra Col'];
  const cm = mapColumns_(hdr);
  ok('column map by name (Occupied->idx 2)', cm['Occupied'] === 2);
  ok('column map maps property label', cm['property_name'] === 0);
  ok('column map ignores unknown column', Object.values(cm).indexOf(5) === -1);

  const grid = [['x'], ['Property', 'Units', 'Occupied'], ['A', 1, 1]];
  ok('header found by name (row 1)', findHeaderRow_(grid) === 1);
  ok('keyOf composite', keyOf_(new Date(2026, 7, 28), 'P1') === '2026-08-28|P1');  // local date avoids tz drift

  const msg = results.join('\n');
  Logger.log(msg);
  const failed = results.filter(function (r) { return r.indexOf('FAIL') === 0; }).length;
  SpreadsheetApp.getActive().toast(failed ? (failed + ' test(s) FAILED') : 'All ' + results.length + ' tests passed',
    'Self-tests', 6);
  return msg;
}
```